Entrega N.º 2 — Prototipo, prueba y métricas

BoxBox — Estrategia de neumáticos en Fórmula 1
Inteligencia Artificial Avanzada, UTN FRBA, 2.º cuatrimestre de 2026.
Repositorio: `https://github.com/tomasanchez/boxbox` (rama `main`).

Este documento es un índice, no un informe. Dice **dónde** está cada cosa que la consigna pide, para no tener que buscarla en las 1.700 líneas de `informe.md`. El contenido está ahí; acá está el mapa.

> «…deberán implementar un prototipo como "prueba de concepto" […] utilizando alguna tecnología candidata a través de herramienta/s, framework/s y/o librería/s disponible/s, deberán llevar a cabo la construcción del prototipo y su posterior prueba incluyendo el análisis de las métricas aplicadas para su evaluación.»

El problema, en una línea

Dado un auto, una grilla y un circuito, decidir **cuántas paradas hacer, en qué vueltas y con qué compuesto**. No se predice quién gana.

1. Tecnología candidata

La técnica principal es el **Algoritmo Genético** (Unidad 3 del programa). El simulador Monte Carlo no es la técnica: es el **entorno** que hace de función de aptitud.

Pieza	Herramienta	Por qué
Computación evolutiva	**DEAP 1.4**	Es la biblioteca que la cátedra indica para la Unidad 3
Motor propio de contraste	numpy 2.x	Para medir qué aporta la biblioteca y qué aporta el diseño
Datos de F1	**FastF1 3.8.3**	Vueltas, compuestos, banderas y pit stops oficiales
Cálculo	numpy, pandas, pyarrow	El simulador está vectorizado sobre 1.200 carreras sorteadas
Modelos auxiliares	scikit-learn, LightGBM	Degradación y ritmo
Interfaz	**React 19 + Vite 8 + MUI 9**	Prototipo visual estilo transmisión
Servicio	FastAPI	Arquitectura *cosmic python*, con sus propios ADR
Calidad	uv, ruff, pytest, oxlint, TypeScript	Lint y tests en CI

Se implementaron dos motores evolutivos sobre la misma función de aptitud —DEAP y uno propio— y se compararon. Esa comparación está en el informe, §5, y no es decorativa: destapó que `eaMuPlusLambda` con `selTournament` **no es elitista** y que el salón de la fama se estaba calculando y descartando.

2. Construcción del prototipo

Qué	Dónde
Simulador de carrera + AG	`apps/ml/src/boxbox_ml/strategy.py`
Motor DEAP	`apps/ml/src/boxbox_ml/deap_search.py`
Política de rivales reactivos	`apps/ml/src/boxbox_ml/reaction.py`
Neutralizaciones y banderas	`apps/ml/src/boxbox_ml/neutralisation.py` , `track_status.py`
Ingesta y features	`apps/ml/src/boxbox_ml/cache.py` , `features.py`
60 scripts de medición	`apps/ml/scripts/`
Interfaz	`apps/web/src/`
Decisiones de diseño	`decisions/boxbox/` (ADR)

Narrativa de construcción: **informe §4 (Desarrollo)**.

3. La prueba

Hay dos clases de prueba y conviene no confundirlas.

Pruebas automatizadas

```
cd apps/ml && uv run pytest -q          # suite
cd apps/ml && uv run ruff check . && uv run ruff format --check .
cd apps/web && npm run lint && npm run build
```

143 tests, que corren en 1,7 segundos. Cubren el núcleo del simulador:

Archivo	Qué prueba	Tests
`test_strategy_repair.py`	El reparador: `MIN_STINT`, `MAX_STINT`, orden de paradas, paradas demasiado tardías	60
`test_strategy_distancia.py`	Que un plan desde la parrilla sume **exactamente** las vueltas de la carrera	21
`test_strategy_dos_compuestos.py`	La obligación reglamentaria B6.3.8	15
`test_strategy_chequered.py`	Cuántas vueltas completa cada auto cuando cae la bandera	14
`test_strategy_costo_parada.py`	El costo de parar según la bandera encontrada	11
`test_strategy_neutralizaciones.py`	El sorteo de banderas y la identidad de los períodos	11
`test_strategy_instrumentation.py`	Que instrumentar el AG no cambie el resultado	11

Dos de ellos son **regresiones de errores reales y caros** que este trabajo ya se comió: que la carrera duraba 71 vueltas de 72, y que vueltas abajo es `floor(hueco / vuelta verde)` y no una resta de segundos.

La suite encontró dos defectos latentes, y quedaron sin arreglar a propósito, marcados con `xfail` estricto para que se pongan en rojo el día que alguien los corrija:

1. `race_trace` documenta que `flags=None` corre todo en verde, pero `_pit_cost` hace `flags.shape` sin mirar si es `None`: cualquier plan **con paradas** revienta. Hoy no lo sufre nadie porque todos los llamadores pasan banderas.
2. `_enforce_two_compounds` corre **después** del control de presupuesto y le cambia el compuesto a la última parada sin volver a mirarlo, así que puede devolver una tanda final pasada del tope de vida — `H35-M37` en una carrera de 72, con el medio topado en 31. Medido: **la búsqueda no lo alcanza**, 0 de 36 corridas devuelven un plan que se pase, porque una tanda así está demasiado degradada para ganar la aptitud. No mueve ninguna cifra publicada.

Que la suite haya encontrado dos bugs el mismo día que se escribió es el argumento a favor de tenerla.

Prueba empírica contra carreras reales

Es la que de verdad evalúa el sistema, y son cuatro:

Prueba	Qué pone a prueba	Dónde
Monza 2026	El sistema completo, cuatro fechas después del circuito base	§6
Madrid 2026	Un circuito sin datos del año en curso	§6, `scripts/madrid_2026_review.py`
Elección de compuesto	Contra una grilla real de 22 autos	§6
Bakú 2026	Un pronóstico **pre-registrado**	§6, `scripts/baku_2026_review.py`

La de Bakú es metodológicamente la más limpia y conviene explicar por qué. Las otras tres se corrieron sabiendo el resultado; aunque nadie haga trampa, el analista elige qué mirar *después*, y eso infla cualquier acierto. Para Bakú se cargó la clasificación, **no** la carrera, se escribieron criterios de falsación explícitos y se commiteó la predicción (`418e90f`). Recién entonces se miró el resultado.

Salió **uno de cuatro**. El análisis de por qué falló vale más que el marcador, y destapó un límite estructural del modelo que no estaba declarado.

4. Las métricas y su análisis

Métrica	Resultado
Error del modelo de degradación	Mejor que la mediana
Ventaja del AG sobre reglas enunciables	**+8,6 a +10,9 s** de carrera
Planes ilegales generados (B6.3.8)	**Cero**
¿La estrategia real cayó en el rango?	Probado en Monza
Criterios de falsación acertados	1 de 4 en Bakú
DEAP contra motor propio	§5

La exactitud no se usa como métrica, y el motivo está medido: «no parar nunca» acierta el 96,6% de las vueltas, y además de inútil es ilegal.

Análisis completo en **§6**, que ocupa la mayor parte del informe. Incluye los **errores de método propios** que se encontraron y corrigieron —cada uno daba números confiados y plausibles hasta que algo aguas abajo salió absurdo— y los **cinco límites estructurales**: cosas que el modelo no tiene dónde representar y que ninguna cantidad de datos arregla.

5. Qué NO está en esta entrega

Decirlo explícitamente, porque el informe lo declara y no conviene que parezca un olvido:

- **El agente de Aprendizaje por Refuerzo.** Es la comparación propuesta —planificar contra reaccionar— y corresponde a la Unidad 5, que se dicta para la **Entrega 3**. El entorno (`gymnasium`) ya está entre las dependencias.
- **La asignación de neumáticos disponibles.** Sin ella la recomendación no es ejecutable para el 85% de los autos. El sistema es honesto dentro de ese techo y lo declara.
- **El escalón de ritmo entre compuestos** (`COMPOUND_OFFSET_S` vale cero). Es una falla de identificación declarada, y es la limitación más cara del trabajo: está contrastada contra una grilla real en §6.

Cómo correrlo

```
# Simulador y algoritmo genético
cd apps/ml && uv sync
uv run python scripts/prerace_strategy.py      # plan pre-carrera
uv run python scripts/deap_vs_builtin.py       # los dos motores
uv run python scripts/baku_2026_review.py      # el pronóstico contra la realidad

# Interfaz
cd apps/web && npm install && npm run dev
```

Los scripts usan la caché de FastF1 en modo desconectado; la primera corrida de un circuito nuevo baja datos y está limitada a 500 llamadas por hora.